



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	M Rezki Riopaldy
NPM	2428240117
Kelas	SI5C
Topik Tugas 1	Kepegawaian: Karyawan
Entitas utama	Karyawan
Tautan repositori	https://github.com/rezkiriopaldy-secret-V/si5c_kuis1_mrezkiriopaldy
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `employees` dan `Karyawan`, dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, `employees` menjadi `buku`.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama `PAW2_Kuis2_SI5C_2428240117_M Rezki Riopaldy.pdf`, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau <code>curl</code> di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan `Win + Shift + S`; macOS tekan `Cmd + Shift + 4`; Linux gunakan `PrtSc` atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan

Bab	Isi	Bukti utama
Bab 3	Menjalankan server	1 screenshot terminal
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Kepegawaian: Karyawan
Entitas utama (tunggal)	Karyawan
Alamat dasar	/employees
Field data	nip (string), nama (string), jabatan (string), departemen (string), gaji (number)
Field wajib saat POST	nip, nama, jabatan, departemen
Query string untuk filter	departemen, contoh: /employees?departemen=Keuangan
Data awal	Agus Pratama (EMP-0045, Staf Akuntansi, Keuangan, gaji 6500000); Siti Rahma; Budi Santoso
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/employees	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/employees/:id	Tidak	200	Ambil satu data
3	POST	/employees	Ya	201	Tambah data
4	PUT	/employees/:id	Ya	200	Ubah data
5	DELETE	/employees/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

```

PS C:\Users\acer\OneDrive\Documents\rio> node -v
v24.21.0
PS C:\Users\acer\OneDrive\Documents\rio> npm -v
11.19.0
PS C:\Users\acer\OneDrive\Documents\rio> npm i express
added 94 packages, and audited 95 packages in 3s
33 packages are looking for funding
run 'npm fund' for details
3 high severity vulnerabilities
To address all issues (including breaking changes), run:
  npm audit fix --force
Run 'npm audit' for details.
PS C:\Users\acer\OneDrive\Documents\rio> npm install --save-dev nodemon
up to date, audited 95 packages in 2s
33 packages are looking for funding
run 'npm fund' for details
3 high severity vulnerabilities
To address all issues (including breaking changes), run:
  npm audit fix --force
Run 'npm audit' for details.
PS C:\Users\acer\OneDrive\Documents\rio> npm init -y
Wrote to C:\Users\acer\OneDrive\Documents\rio\package.json:

{
  "name": "tugas1-restful-2428240117",
  "version": "1.0.0",
  "description": "RESTful API sederhana menggunakan Express.js untuk mengelola data Karyawan.",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.21.0",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5"
  },
  "devDependencies": {
    "nodemon": "^3.1.7"
  }
}
  
```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

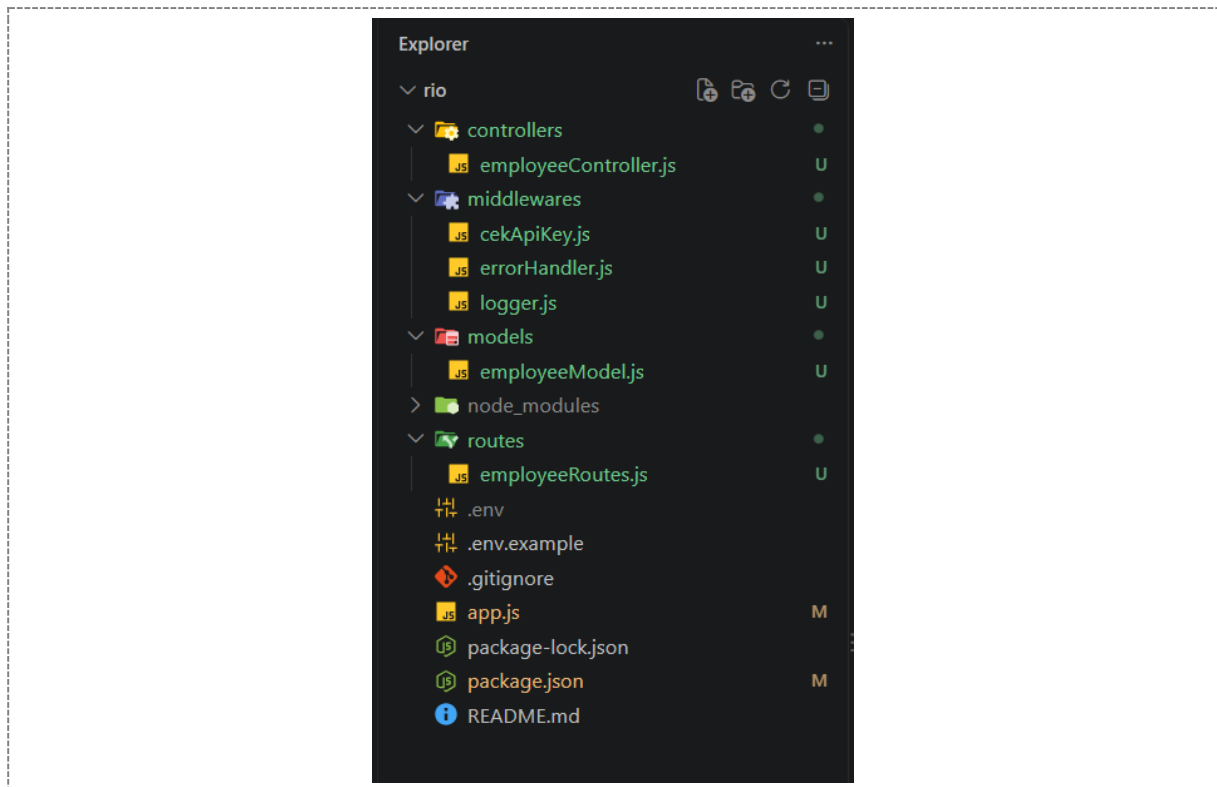
Express digunakan sebagai framework untuk membuat server dan route RESTful API, cors memungkinkan API menerima request dari origin lain, sedangkan dotenv membaca konfigurasi dari file `.env`. Node.js 24.21.0 digunakan pada project ini dan mendukung kebutuhan runtime Express 5.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Folder routes memetakan endpoint ke controller, controllers menangani request/validasi/response, models menyimpan data dan fungsi pengolahannya, sedangkan middlewares berisi fungsi yang dijalankan di antara request dan handler seperti logger, API key, dan error handler.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

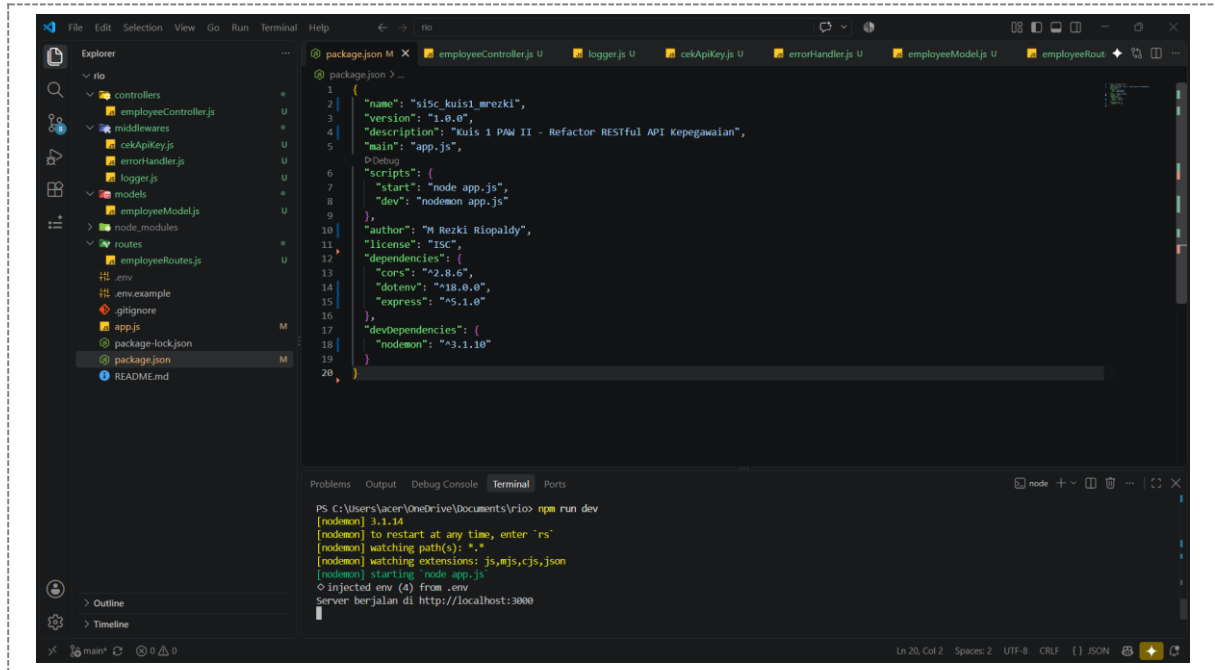
Berkas	Penjelasan Anda
models/employeeModel.js	Menyimpan array employees serta fungsi getAllEmployees, getEmployeeById, createEmployee, updateEmployee, dan deleteEmployee. Model tidak menggunakan req atau res sehingga fokus pada data dan pengolahannya.
controllers/employeeController.js	Berisi getEmployees, getEmployeeById, createEmployee, updateEmployee, dan deleteEmployee untuk membaca request, melakukan validasi, memanggil Model, lalu mengirim response.

Berkas	Penjelasan Anda
routes/employeeRoutes.js	Menggunakan <code>express.Router()</code> untuk memetakan GET, POST, PUT, dan DELETE pada <code>/employees</code> ke fungsi Controller; POST, PUT, dan DELETE diberi middleware <code>cekApiKey</code> .
middlewares/logger.js	Mencatat waktu, method, dan URL setiap request melalui <code>console.log</code> sebelum request diteruskan dengan <code>next()</code> .
middlewares/cekApiKey.js	Membaca header <code>x-api-key</code> dan membandingkannya dengan <code>process.env.API_KEY</code> ; request tanpa atau dengan key yang salah ditolak dengan status 401.
middlewares/errorHandler.js	Menangani rute yang tidak ditemukan melalui <code>notFoundHandler</code> dan mengubah JSON parse error maupun error lain menjadi response JSON terpusat.
app.js	Memuat <code>dotenv</code> , <code>cors</code> , <code>express.json()</code> , <code>logger</code> , route <code>/employees</code> , <code>notFoundHandler</code> , dan <code>errorHandler</code> , kemudian menjalankan server pada PORT dari <code>.env</code> .

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port berasal dari `process.env.PORT` pada file `.env` dengan nilai 3000. Jika port 3000 sedang dipakai aplikasi lain, server tidak dapat melakukan listen pada port tersebut dan Node.js akan menghasilkan error sehingga aplikasi perlu memakai port lain atau menghentikan proses yang menggunakan port tersebut.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

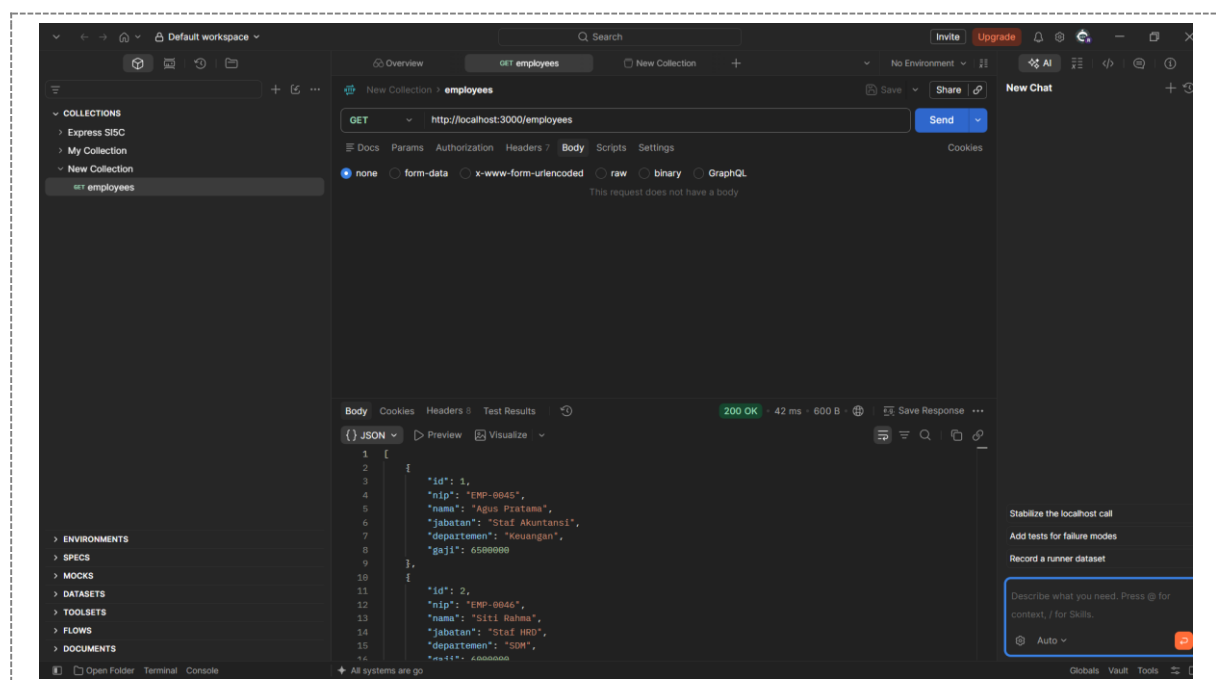
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /employees)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (request)	GET /employees
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK — data karyawan ditampilkan.
Lulus?	[✓] Ya [] Tidak



Gambar 4. Hasil GET semua data

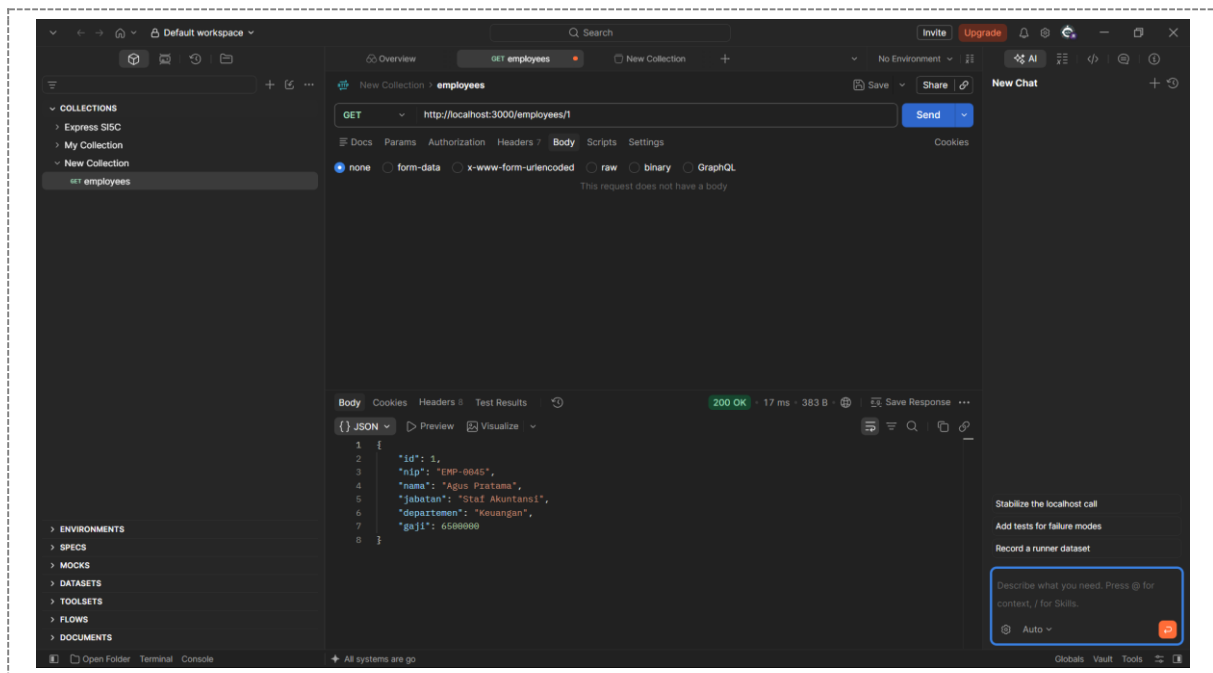
Penjelasan:

Request GET /employees masuk ke employeeRoutes melalui route GET '/', lalu menjalankan getEmployees pada employeeController. Controller mengambil query departemen jika ada dan memanggil getAllEmployees pada employeeModel; hasilnya dikirim dengan status 200 OK.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /employees/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id 1.
Status hasil uji	200 OK — data dengan id 1 ditemukan.
Lulus?	☒ Ya ☐ Tidak



Gambar 5. Hasil GET berdasarkan ID

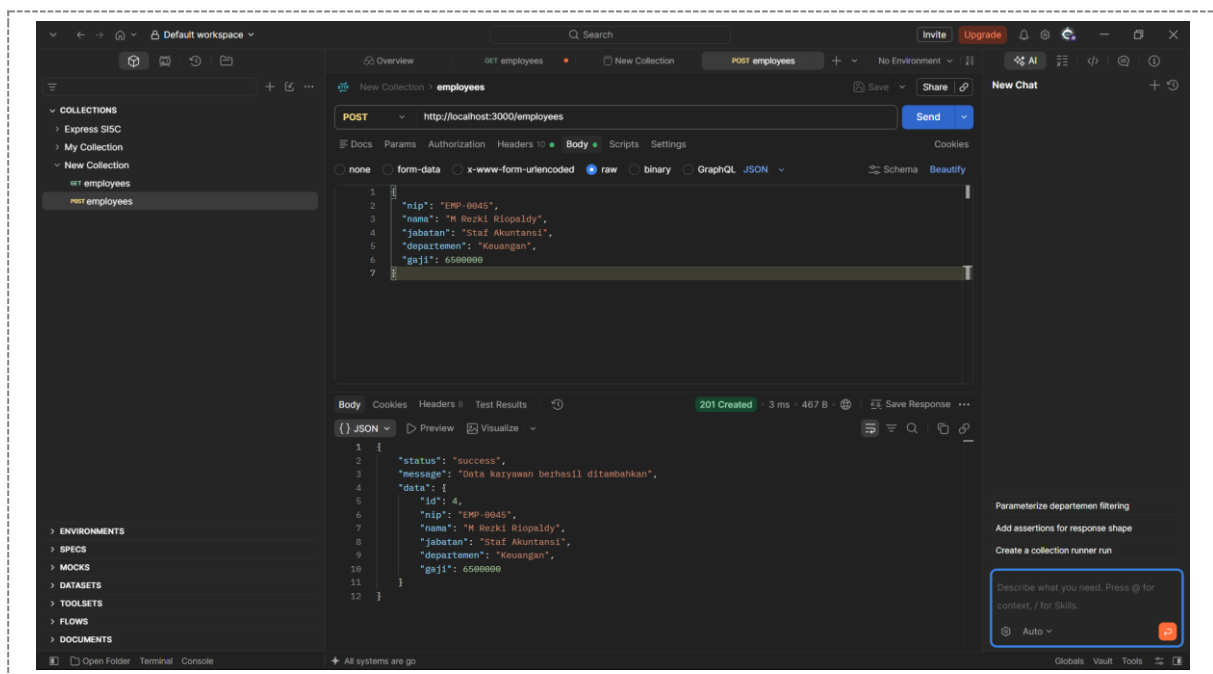
Penjelasan:

Nilai ID diambil dari req.params.id karena ID berada pada bagian parameter URL, misalnya /employees/1. ID kemudian diubah menjadi angka dengan Number(id) agar dapat dibandingkan dengan properti id yang bertipe number pada data model.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /employees
Header	x-api-key: SI5C-KUIS1-2428240117 Content-Type: application/json
Body	<pre>{ "nip": "EMP-0045", "nama": "M Rezki Riopaldy", "jabatan": "Staf Akuntansi", "departemen": "Keuangan", "gaji": 6500000 }</pre>
Hasil yang diharapkan	201 Created; objek baru beserta id 4 yang dibuat server.
Status hasil uji	201 Created — Data karyawan berhasil ditambahkan.
Lulus?	☒ Ya ☐ Tidak



Gambar 6. Hasil POST data baru

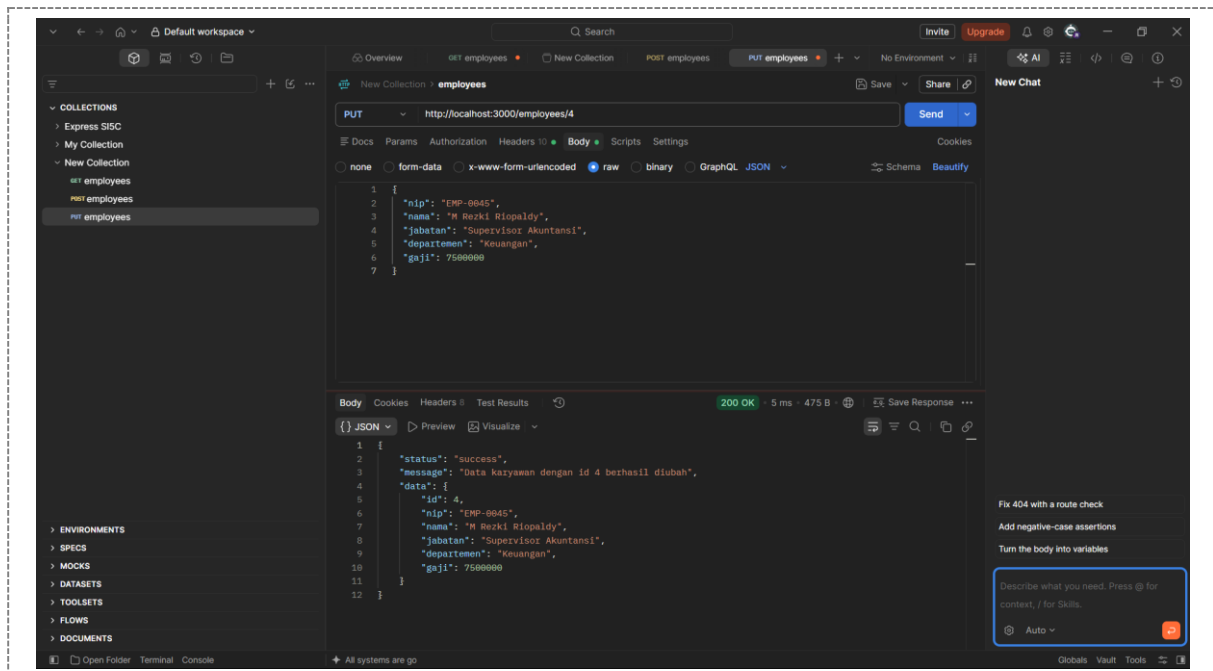
Penjelasan:

Header x-api-key diperiksa oleh middleware cekApiKey sebelum controller dijalankan. `express.json()` mengubah body JSON menjadi `req.body` agar dapat divalidasi dan diproses, sedangkan status 201 dipakai karena request berhasil membuat resource baru.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /employees/4
Header	x-api-key: SI5C-KUIS1-2428240117 Content-Type: application/json
Body	{"nip":"EMP-0045","nama":"M Rezki Riopaldy","jabatan":"Supervisor Akuntansi","departemen":"Keuangan","gaji":7500000}
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK — Data karyawan dengan id 4 berhasil diubah.
Lulus?	[✓] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

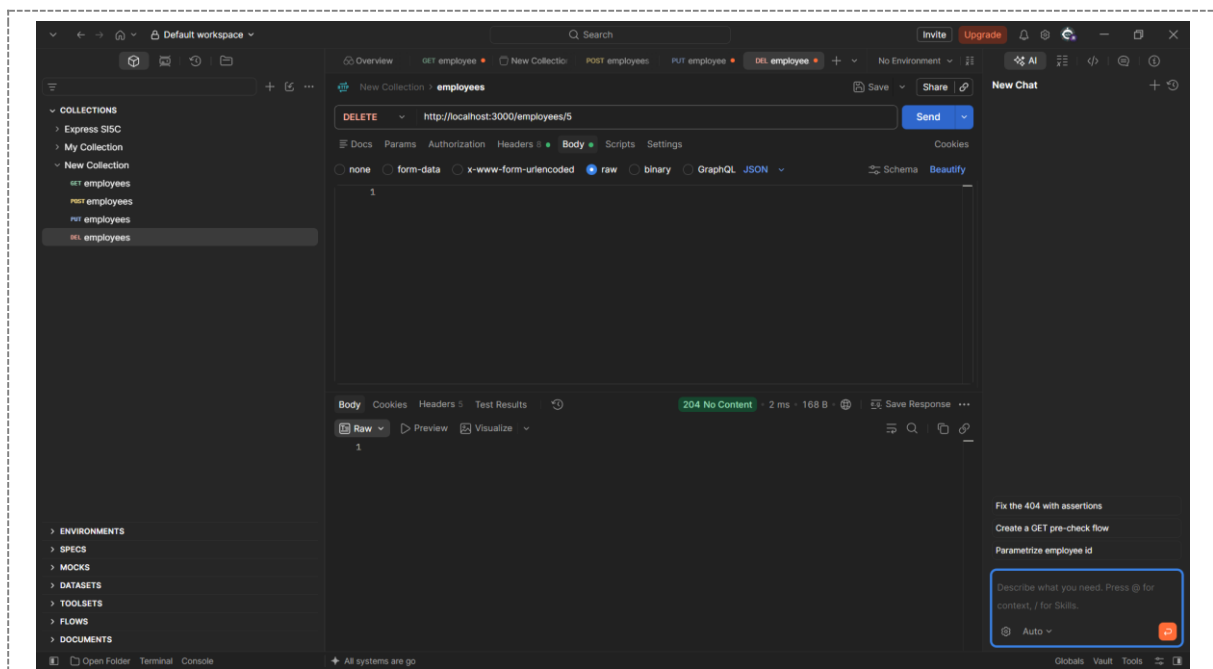
Penjelasan:

Pada T04, jabatan berubah dari "Staf Akuntansi" menjadi "Supervisor Akuntansi" dan gaji berubah dari 6500000 menjadi 7500000, sedangkan NIP, nama, dan departemen tetap. Perubahan data dilakukan pada layer Model melalui fungsi `updateEmployee`, sedangkan Controller mengatur validasi dan response.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /employees/5
Header	x-api-key: SI5C-KUIS1-2428240117
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content — data berhasil dihapus tanpa isi respons.
Lulus?	[✓] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Status 204 No Content menunjukkan penghapusan berhasil tetapi server tidak mengirim isi response. Karena resource sudah berhasil dihapus, tidak ada data yang perlu dikembalikan pada body.

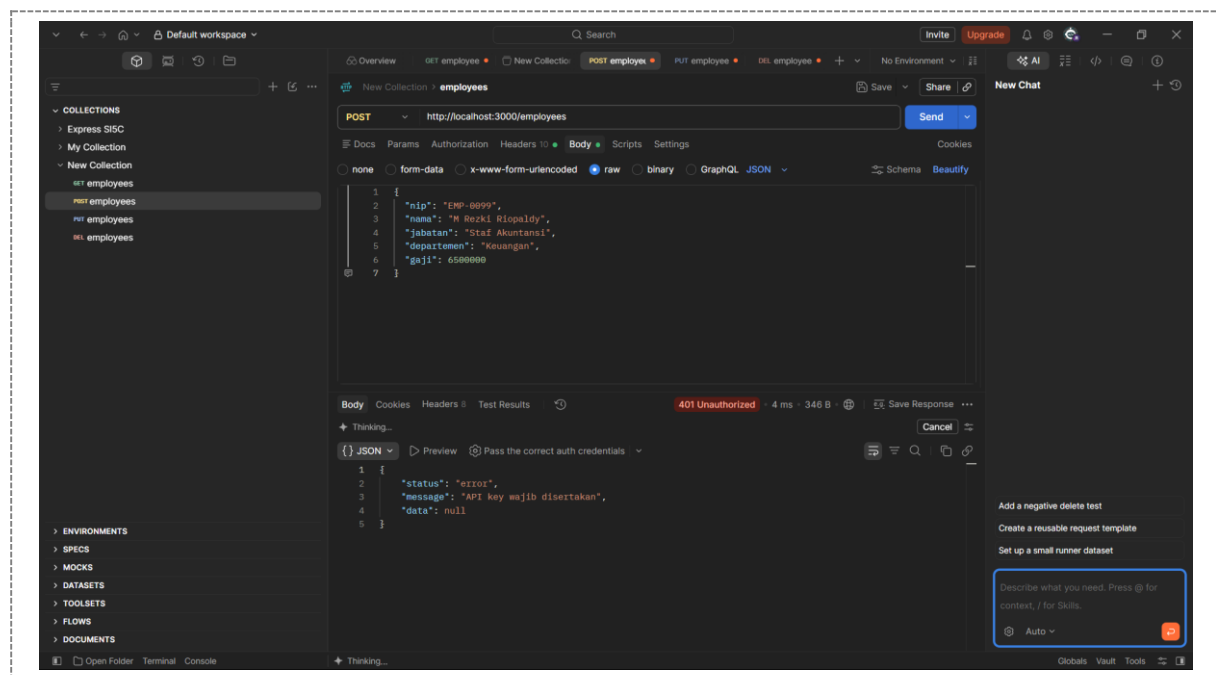
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada **screenshot**: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /employees (tanpa x-api-key)
Header	Content-Type: application/json
Body	<pre>{ "nip": "EMP-0099", "nama": "M Rezki Riopaldy", "jabatan": "Staf Akuntansi", "departemen": "Keuangan", "gaji": 6500000 }</pre>
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized — API key wajib disertakan.
Lulus?	[✓] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

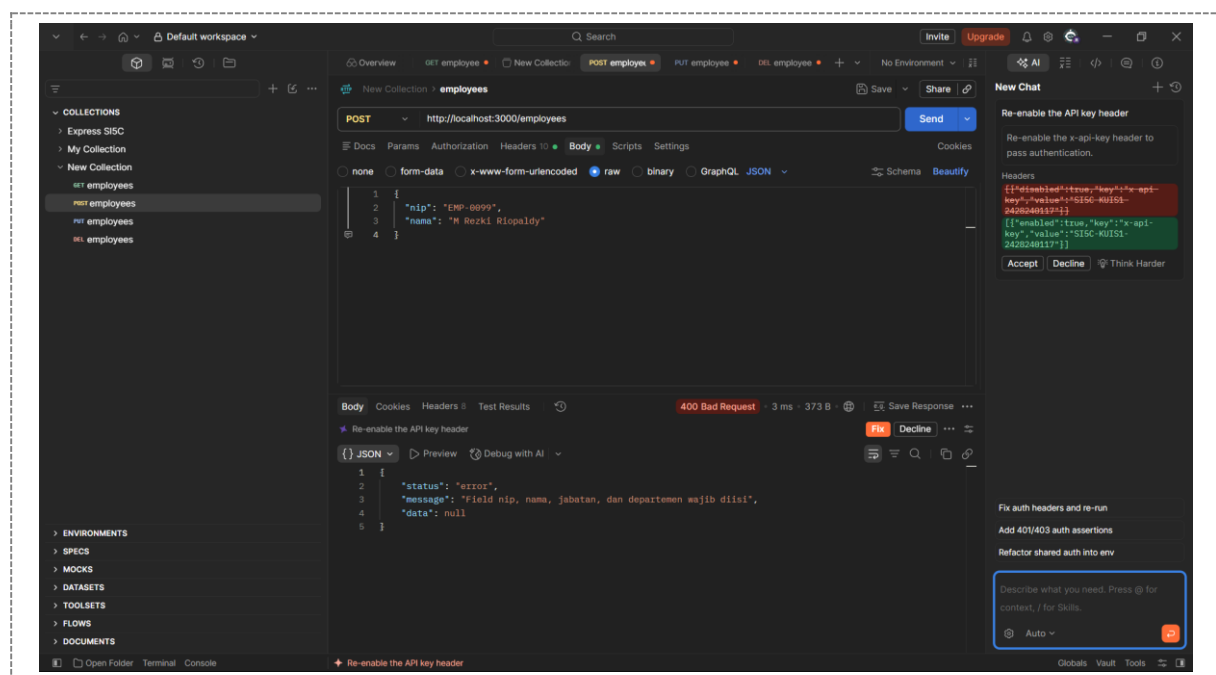
Penjelasan:

Permintaan ditolak oleh middleware cekApiKey karena header x-api-key tidak disertakan. Middleware langsung mengembalikan status 401 sehingga Controller createEmployee tidak sempat dijalankan.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /employees
Header	x-api-key: SI5C-KUIS1-2428240117 Content-Type: application/json
Body	{"nip":"EMP-0099","nama":"M Rezki Riopaldy"}
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request — Field nip, nama, jabatan, dan departemen wajib diisi.
Lulus?	[✓] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

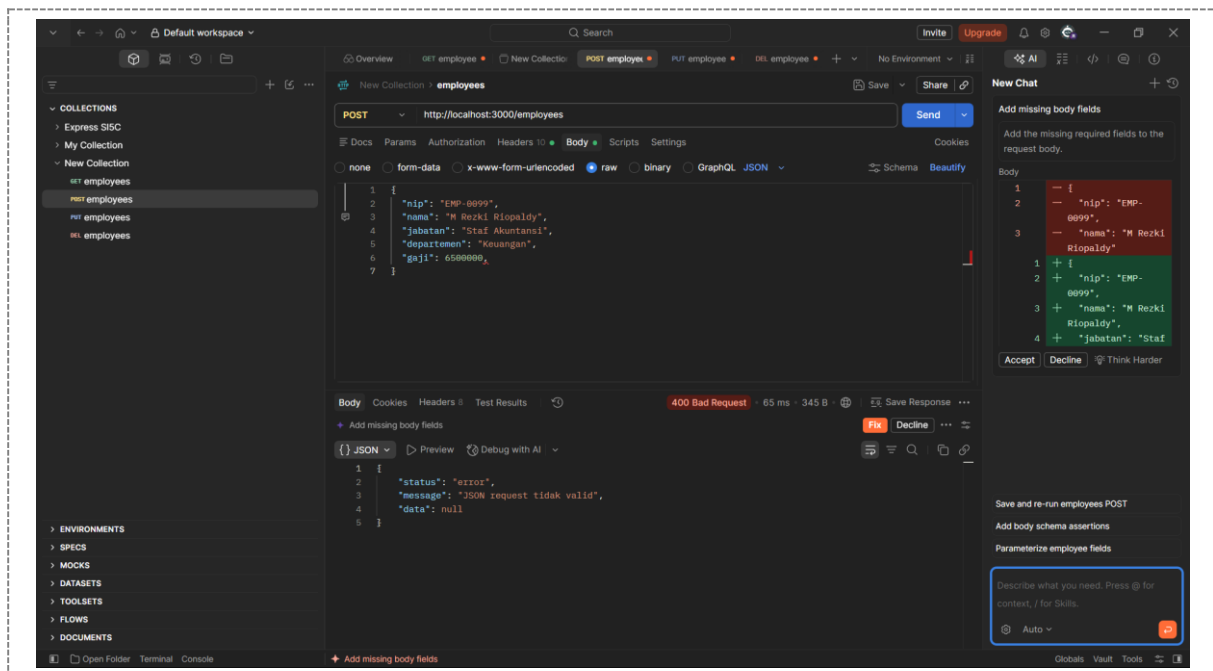
Penjelasan:

Validasi body tidak lengkap ditulis pada Controller `createEmployee`, setelah request melewati middleware API key. Jika nip, nama, jabatan, atau departemen tidak ada, Controller mengembalikan 400 Bad Request karena data yang dikirim tidak memenuhi field wajib.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /employees
Header	x-api-key: SI5C-KUIS1-2428240117 Content-Type: application/json
Body	{ "nip": "EMP-0099", "nama": "M Rezki Riopaldy", }
Hasil yang diharapkan	400 Bad Request; JSON request tidak valid.
Status hasil uji	400 Bad Request — JSON request tidak valid.
Lulus?	[✓] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

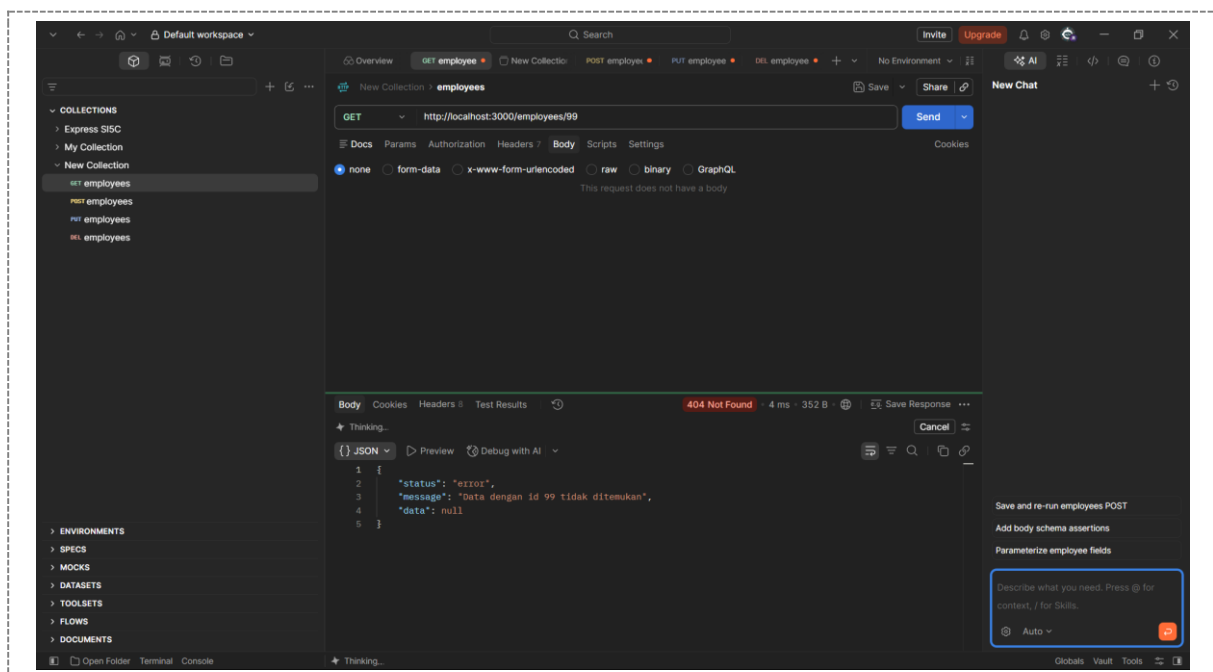
Penjelasan:

JSON yang rusak dideteksi oleh `express.json()` saat proses parsing body. Error tersebut kemudian diteruskan ke `errorHandler`, yang mengenali `err.type entity.parse.failed` dan mengubahnya menjadi response JSON 400 dengan pesan "JSON request tidak valid".

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /employees/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found — Data dengan id 99 tidak ditemukan.
Lulus?	☒ Ya ☐ Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Request /employees/99 masuk ke getEmployeeById pada Controller, lalu Controller memanggil employeeModel.getEmployeeById. Karena model tidak menemukan data dengan ID 99, Controller mengirim response 404 dengan pesan "Data dengan id 99 tidak ditemukan".

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	OK	YA
T02	Menampilkan satu data berdasarkan ID	200	OK	YA
T03	Menambah data baru	201	Created	YA
T04	Mengubah data	200	OK	YA
T05	Menghapus data	204	No Content	YA
N01	Menambah data tanpa API key	401	Unauthorized	YA
N02	Menambah data dengan body tidak lengkap	400	Bad Request	YA
N03	Mengirim JSON yang rusak	400	Bad Request	YA
N04	Mengambil ID yang tidak ada	404	No Found	YA

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/rezkiriopaldy-secret-V/si5c_kuis1_mrezkiriopaldy
Pesan commit terakhir	feat: add structured employee REST API
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route bertugas memetakan alamat API, misalnya `router.get('/', getEmployees)` pada `employeeRoutes.js`. Controller menangani request seperti `getEmployees`, sedangkan Model mengolah data melalui `getAllEmployees`; pemisahan ini membuat setiap layer memiliki tanggung jawab yang jelas.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Error handler terpusat membuat penanganan error lebih konsisten tanpa mengulang format response pada setiap route. Buktinya, JSON rusak pada N03 dikonversi menjadi 400 JSON oleh errorHandler, sedangkan ID 99 pada N04 menghasilkan 404 JSON dari Controller ketika data tidak ditemukan.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	✓
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	✓
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	✓
4	Identitas saya (nama atau NIM) muncul pada data uji POST	✓
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	✓
6	Tabel rekapitulasi sesuai dengan tangkapan layar	✓
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	✓
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	✓

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100